

AI-Driven Personal Protective Equipment (PPE) Compliance Monitoring System for Construction Safety

Author: Parth Mital, 23BCE0842

Affiliation: Vellore Institute of Technology, Vellore

Repository: <https://github.com/parthmital/PPE-Compliance-Monitor>

Abstract

The construction industry in India is among the most hazardous sectors, accounting for a disproportionately high number of fatal and non-fatal occupational accidents. A primary causative factor is the non-compliance with Personal Protective Equipment (PPE) protocols, including hardhats, safety vests, and face masks. Traditional manual supervision methods are labour-intensive, error-prone, and incapable of providing continuous, real-time monitoring across large, dynamic construction sites. This paper presents the development, training, and deployment of an end-to-end, real-time PPE compliance monitoring system leveraging state-of-the-art deep learning and computer vision techniques. The system is architected around a fine-tuned YOLOv8m object detection model, trained on a curated dataset of 2,801 images comprising 38,352 annotated instances across ten distinct classes, including critical PPE items and their corresponding violations. The model achieves a mean Average Precision ([mAP@0.5](#)) of 88.07% on a held-out test set, demonstrating excellent detection capability. This detection engine is seamlessly integrated into a full-stack web application featuring an asynchronous FastAPI backend for robust media processing and a modern React-based frontend for real-time visualisation, safety scoring, incident logging, and session persistence. The system successfully balances high detection accuracy with real-time performance (approximately 40 frames per second on suitable hardware), offering a scalable, effective, and practical solution for enhancing occupational safety on dynamic construction sites across India.

Keywords: Construction Safety, Personal Protective Equipment (PPE), Deep Learning, YOLOv8, Computer Vision, Real-Time Detection, Indian Construction Industry, FastAPI, React.

1. Introduction

The safety and well-being of personnel on construction sites is a matter of paramount national importance, particularly given the rapid urbanisation and infrastructure development underway across India. The construction sector is a significant pillar of the Indian economy, employing nearly 10 million individuals, making it the second-largest employer after agriculture [1]. However, this scale of employment is matched by a sobering safety record. The industry contributes to approximately 24.2% of all workplace fatalities, with an estimated average of 38 fatal accidents occurring every single day. Annually, between 11,614 and 22,080 occupational deaths are reported across the country, with the construction sector consistently accounting for the highest share of these fatalities [2].

A thorough analysis of these accidents reveals a persistent and troubling trend: the majority of incidents, including falls from heights, struck-by-object injuries, and electrocutions, are directly attributable to the non-use or improper use of Personal Protective Equipment (PPE). Hardhats, safety vests, gloves, and more recently, face masks (due to ongoing pandemic and air quality concerns), serve as the last line of defence against workplace hazards. Despite clear regulatory mandates and safety protocols, a culture of non-compliance persists on many sites due to factors such as worker fatigue, lack of awareness, inadequate supervision, and a perceived discomfort in wearing the equipment.

Conventional safety monitoring methods, which rely on manual visual inspections by site engineers and safety officers, are fundamentally flawed for several reasons:

1. **Limited Coverage:** A human supervisor cannot be everywhere at once, leaving significant portions of a large construction site unmonitored.
2. **Human Error:** Fatigue, distraction, and cognitive biases can cause even diligent supervisors to miss critical safety violations.
3. **Lack of Persistence:** Manual monitoring is episodic, failing to provide continuous, 24/7 surveillance.
4. **Post-Event Analysis:** Without a persistent record, it is difficult to analyse trends, identify high-risk zones, or conduct effective post-accident investigations.

To overcome these limitations, there is an urgent and critical need for automated, intelligent, and real-time monitoring systems. Recent advancements in the field of deep learning, specifically in computer vision-based object detection, offer a powerful and transformative solution to this challenge. Convolutional Neural Networks (CNNs), and particularly the "You Only Look Once" (YOLO) family of algorithms, have demonstrated remarkable success in identifying and localising objects within images and video streams with high speed and accuracy. These models can be trained to recognise PPE items and instantly flag violations, providing a proactive, data-driven approach to safety management.

This research paper presents the development of a complete, production-ready PPE compliance monitoring system tailored for the unique challenges of the Indian construction landscape. The primary contributions of this work are as follows:

1. **Development of a High-Performance Detection Model:** A YOLOv8m object detection model was meticulously trained and fine-tuned on a large, diverse dataset to accurately identify ten distinct classes, including Hardhat, Mask, Safety Vest, and their corresponding violation classes (NO-Hardhat, NO-Mask, NO-Safety Vest), alongside other relevant objects like Person, machinery, and vehicle. The model achieves a state-of-the-art [mAP@0.5](#) of 88.07%.
2. **Engineering of a Robust, Full-Stack Application:** A complete web-based system was architected and implemented, comprising:
 - **Asynchronous Backend:** A FastAPI backend that handles image and video uploads, performs asynchronous video processing to prevent blocking, and provides a comprehensive set of RESTful APIs for configuration, detection, and data retrieval.
 - **Interactive Frontend:** A modern, responsive React frontend that provides a real-time dashboard for visualising detections, a dynamic safety score, key performance metrics, an incident log, and an intuitive settings panel for adjusting model thresholds.
3. **Implementation of Practical Enhancements:** The system incorporates several practical features, including a temporal buffer to reduce false positive alerts, complete data persistence (incidents, metrics, and UI state survive server restarts), and the ability to hot-swap the detection model at runtime.
4. **Benchmarking and Applicability:** The system's performance is rigorously evaluated using standard metrics and benchmarked against findings from recent literature. The solution is designed for practical deployment by construction contractors, particularly those operating in the Ambur–Vellore region of Tamil Nadu, a major hub for construction and leather processing industries.

The remainder of this paper is structured as follows: Section 2 provides a comprehensive literature survey of related work in computer vision-based safety monitoring. Section 3 details the deep learning algorithms and YOLOv8 architecture used. Section 4 outlines the hardware and software requirements for the system. Section 5 presents the experimental analysis, including dataset preparation, training methodology, and evaluation metrics. Section 6 discusses the results and

benchmarks the system against prior research. Finally, Section 7 concludes the paper and outlines directions for future work.

2. Literature Survey

The application of computer vision, particularly deep learning-based object detection, for automated safety monitoring in construction has been an active area of research. This section reviews seminal and recent contributions in this domain, tracing the evolution from earlier architectures to the state-of-the-art.

2.1 Early Implementations with YOLOv3 and YOLOv4

Initial forays into PPE detection leveraged the then-groundbreaking YOLOv3 and YOLOv4 architectures. Shi et al. [3] demonstrated the feasibility of using YOLOv3 for helmet detection by employing an Image Pyramid to extract feature maps at various scales, improving detection across different object sizes. Addressing the specific challenge of mask detection during the COVID-19 pandemic, Wei et al. [4] proposed an enhanced YOLOv3 model named "Face_mask Net." This model integrated Spatial Pyramid Pooling (SPP) and Cross-Stage Partial Network (CSPNet) modules to improve feature extraction and used a Distance Intersection over Union (DIOU) loss function and DIOU-NMS algorithm to better handle occlusions and small targets. Their work showed significant improvements in detection precision for masked faces in public places.

Cheng et al. [5] focused on the need for speed and lightweight deployment by enhancing the YOLOv4-tiny model for mask detection. They replaced the CSP module with two Resblock-D modules to reduce network complexity and incorporated a two-layer Coordinate Attention (CA) mechanism. This approach successfully improved detection speed while maintaining high accuracy, making it suitable for deployment on resource-constrained edge devices. Similarly, Xie et al. [6] proposed an improved YOLOv4 algorithm, SMD-YOLOv4, for helmet detection in complex backgrounds. Their work demonstrated that architectural optimisations could enable the model to capture more target features without sacrificing inference speed, a critical requirement for real-world construction sites with cluttered and dynamic backgrounds.

2.2 Advancements and Trade-offs with YOLOv5

YOLOv5 emerged as a highly popular choice due to its excellent balance between speed, accuracy, and ease of use. Song et al. [7] incorporated the CoordAtt coordinate attention mechanism into the YOLOv5s backbone to allocate more attention to small helmet targets, addressing the common problem of missed detections for workers at a distance. They also replaced residual blocks with Res2NetBlock structures to improve fine-grained information fusion, leading to better detection of helmets in dense crowds.

A pivotal comparative study by Wang et al. [8] provided a comprehensive benchmark of YOLOv3, YOLOv4, and YOLOv5 models on a challenging, real-world dataset called CHV (Construction Hard Hat and Vest). This dataset comprised 1,330 images captured from actual construction sites, featuring varied backgrounds, lighting conditions, occlusions, and worker postures. Their results showed that while YOLOv5x achieved the highest mean Average Precision (mAP) of 86.55%, YOLOv5s was the fastest, processing 52 frames per second (FPS) on a GPU. This study highlighted the inherent trade-off between model complexity, speed, and accuracy, and underscored the critical importance of using high-quality, representative datasets for training.

2.3 Lightweight Architectures and Attention Mechanisms

Recognising the need for models that could be deployed on edge devices like CCTV cameras or drones, researchers explored lightweight architectures. Li et al. [9] presented a highly effective approach by improving the YOLOv5s network for simultaneous detection of safety helmets and masks. Their key contributions were:

- **Lightweight Backbone:** Replacing the standard YOLOv5s backbone with ShuffleNetv2, a convolutional neural network (CNN) architecture designed specifically for efficiency on mobile and embedded devices. This drastically reduced the model's parameter and computational loads.
- **Efficient Attention:** Integrating the Efficient Channel Attention (ECA) mechanism, which learns channel attention without performing dimensionality reduction (unlike the Squeeze-and-Excitation (SE) mechanism), thereby avoiding information loss and reducing computational overhead.
- **Optimised Loss Function:** Modifying the loss function to use Efficient Intersection over Union (EIoU) loss, which provides a more accurate penalty term for bounding box regression by separately considering the overlap, centre distance, and aspect ratio.

Their experimental results were impressive: the improved model achieved a 4.3% increase in mAP while simultaneously reducing parameters by 54.8% and computational load by 53.8%, striking an ideal balance between precision and lightweight operation.

2.4 The State-of-the-Art: YOLOv8 and Comprehensive Comparisons

The latest evolution in the YOLO series, YOLOv8, represents a significant leap in performance. Singha et al. [10] demonstrated a real-time PPE detection system using YOLOv8, trained on a 10-class dataset from Roboflow, achieving a practical accuracy of 76.3% with high real-time capabilities. They highlighted YOLOv8's superior speed compared to YOLOv5, making it optimal for applications where rapid detection is paramount.

The most exhaustive comparative analysis was conducted by Elesawy et al. [11], who benchmarked ten different YOLOv5 models and five different YOLOv8 models on the CHV dataset. Their rigorous evaluation, using metrics such as precision, recall, F1-score, and mAP, revealed that YOLOv8m outperformed all other models, achieving the highest mAP of 92.30% and an F1-score of 0.89. This represented a substantial 6.64% improvement over previous benchmarks reported in the literature. Their work conclusively demonstrated the superiority of the YOLOv8 architecture, particularly the medium (m) variant, for the task of PPE detection.

2.5 Identification of Research Gap and Project Contribution

While the reviewed literature has made significant strides in algorithmic improvements and model comparisons, a critical gap remains. The vast majority of studies conclude their contribution at the model validation stage. There is a distinct lack of **end-to-end, production-ready systems** that integrate a state-of-the-art detection model with a functional backend API, a user-friendly frontend interface, and practical features for real-world deployment.

This project directly addresses this gap by making the following key contributions:

1. **Benchmarking and Fine-Tuning of YOLOv8m:** We independently train and rigorously evaluate a YOLOv8m model on a large, diverse 10-class dataset (2,801 images, 38,352 annotations), achieving a test [mAP@0.5](#) of 88.07%, confirming its high performance.
2. **Full-Stack System Engineering:** We architect and implement a complete web application with an asynchronous FastAPI backend (for non-blocking video processing) and a modern, interactive React frontend (for real-time visualisation and analytics).
3. **Practical Feature Implementation:** We incorporate real-world enhancements, including a temporal buffering mechanism to drastically reduce false alarms, complete data persistence to disk (incidents, metrics, UI state), and runtime model hot-swapping.
4. **Documentation for Deployability:** We provide comprehensive documentation, including hardware/software requirements and a quick-start guide, making the system readily

deployable by construction contractors, with specific applicability for the Ambur–Vellore region.

3. Algorithms Used

The core of the proposed PPE detection system is the YOLOv8 object detection algorithm. This section explains the fundamental principles of YOLO, details the specific architectural improvements in YOLOv8, and outlines the training methodology and hyperparameters used to fine-tune the model for our specific task.

3.1 The YOLO Paradigm: Unified, Real-Time Detection

Traditional object detectors, such as the R-CNN family (R-CNN, Fast R-CNN, Faster R-CNN), operate using a two-stage approach. The first stage generates a set of region proposals (candidate bounding boxes where an object might be present). The second stage then classifies each of these proposals into object categories and refines the bounding box coordinates. While accurate, this two-stage process is computationally expensive and slow, making it unsuitable for real-time applications like video surveillance.

The "You Only Look Once" (YOLO) family of algorithms, first introduced by Redmon et al. [12], revolutionised object detection by framing it as a single, unified regression problem. The key innovation is that YOLO looks at the entire image only once. It divides the input image into an $S \times S$ grid. For each grid cell, the algorithm predicts:

- **Bounding Boxes:** A fixed number of bounding boxes, each parameterised by its centre coordinates (x, y), width (w), height (h), and a confidence score indicating the likelihood that the box contains an object.
- **Class Probabilities:** A probability distribution over all possible object classes for the grid cell.

All these predictions are made simultaneously in a single forward pass through a single convolutional neural network. This end-to-end, single-stage architecture is what gives YOLO its exceptional speed, capable of processing over 40-155 frames per second (FPS) depending on the model variant, while still delivering highly competitive accuracy.

3.2 YOLOv8 Architecture: Key Improvements

YOLOv8, the latest major version from Ultralytics, builds upon the successes of YOLOv5 while introducing several architectural refinements that enhance both speed and accuracy. The architecture, as illustrated conceptually in the provided README.md, comprises three main parts: Backbone, Neck, and Head.

- **Backbone (Feature Extraction):** The backbone is responsible for extracting a hierarchical set of features from the input image. YOLOv8 uses a modified version of the CSPDarknet53 (Cross-Stage Partial Darknet-53) architecture. A significant change from YOLOv5 is the replacement of the C3 module with the **C2f (Cross-stage Partial with 2 convolutions and fusing) module**. The C2f module provides richer gradient flow by concatenating the outputs of multiple bottleneck blocks, leading to better feature representation with lower computational cost. This is particularly beneficial for detecting small objects like distant hardhats.
- **Neck (Feature Aggregation):** The neck serves as a bridge, aggregating features from different layers of the backbone to create a robust, multi-scale representation. YOLOv8 continues to use a combination of a **Feature Pyramid Network (FPN)** and a **Path Aggregation Network (PAN)**.

- **FPN** propagates strong semantic features (from deeper, higher-level layers) down the pyramid, which helps in detecting small objects.
- **PAN** complements this by propagating strong localization features (from shallower, lower-level layers) up the pyramid, which helps the model precisely localise objects of all sizes. This FPN-PAN structure ensures that the final feature maps used for prediction contain both rich semantic information and accurate spatial details.
- **Head (Prediction):** The detection head is where YOLOv8 makes its most significant departure from its predecessors. It adopts an **anchor-free** design. Previous YOLO versions relied on a set of pre-defined anchor boxes of various sizes and aspect ratios. The model would then predict offsets relative to these anchors. This approach required careful tuning of anchor box hyperparameters. YOLOv8 predicts the centre of an object directly and then regresses the distance to the four sides of its bounding box. This simplifies the model, reduces the number of hyperparameters, and improves generalisation. Furthermore, the head is **decoupled**, meaning separate branches of the network handle the classification task (determining the object's class) and the regression task (refining the bounding box coordinates). This decoupling leads to faster convergence and higher overall performance.

For this project, the **YOLOv8m (medium)** variant was selected. As demonstrated by the benchmark study of Elesawy et al. [11], YOLOv8m offers the optimal trade-off between high mAP (accuracy) and inference speed, making it the ideal choice for a real-time monitoring system.

3.3 Model Training and Hyperparameter Optimisation

The YOLOv8m model was trained from pre-trained weights (transfer learning) on a custom dataset using the Ultralytics framework. The training was performed on a dual NVIDIA Tesla T4 GPU setup for acceleration. The key hyperparameters, as defined in the Construction-Site-Safety.ipynb notebook, are detailed in Table 1.

Table 1: Key Training Hyperparameters

Parameter	Value	Description
Model Size	m (medium)	YOLOv8m variant.
Epochs	100	Number of complete passes through the training dataset.
Image Size	640 pixels	Input image is resized to 640x640.
Batch Size	32	Number of images processed before model weights are updated (16 per GPU).
Optimizer	AdamW	Adaptive Moment Estimation with weight decay, automatically selected by YOLOv8 for better generalisation.
Initial LR	0.01	Starting learning rate for the optimizer.
Final LR Factor	0.01	Learning rate decays to $lr_0 * lrf = 1e-4$ using a cosine annealing schedule.
Momentum	0.937	Momentum factor for the optimizer.
Weight Decay	0.0005	L2 regularisation parameter to prevent overfitting.

- **Loss Functions:** The YOLOv8 training optimises a composite loss function comprising three components:
 1. **Box Loss (CIoU Loss):** Uses Complete Intersection over Union (CIoU) loss for bounding box regression. CIoU considers three geometric factors: overlap area (IoU),

centre point distance, and aspect ratio, leading to faster and more accurate bounding box convergence.

2. **Class Loss (BCE Loss):** Uses Binary Cross-Entropy (BCE) loss for the multi-label classification task.
 3. **Distributed Focal Loss (DFL):** Helps the network focus on high-quality bounding boxes by increasing the loss contribution from ambiguous predictions.
- **Data Augmentation:** To improve the model's robustness and ability to generalise to unseen real-world scenarios, a suite of online data augmentation techniques was applied during training. These augmentations artificially expand the training dataset by creating modified versions of the original images.
 1. **Geometric:** Mosaic (combining 4 images), Mixup (blending 2 images), Copy-Paste, random rotation ($\pm 5^\circ$), scaling ($\pm 50\%$), translation ($\pm 10\%$), shearing ($\pm 2.0^\circ$), and horizontal flipping (50% probability).
 2. **Photometric:** HSV colour space jittering, adjusting hue ($\Delta=0.015$), saturation ($\Delta=0.7$), and value/brightness ($\Delta=0.4$).

4. Hardware and Software Requirements

The developed system is divided into two primary operational contexts: the model training environment and the production inference and web application environment. The specifications for each are detailed below.

4.1 Model Training Environment

This environment was used to train the YOLOv8m model from scratch. The exact specifications, as captured in the Construction-Site-Safety.ipynb notebook running on Kaggle, are as follows.

- **Hardware:**
 - **GPU:** $2 \times$ NVIDIA Tesla T4 (16 GB VRAM each)
 - **RAM:** Approximately 30 GB
 - **CPU:** Intel Xeon CPU @ 2.00GHz (virtualized)
- **Software & Libraries:**
 - **Operating System:** Ubuntu 18.04.6 LTS (Kaggle base environment)
 - **Language:** Python 3.12.12
 - **Deep Learning Framework:** PyTorch 2.10.0+cu128, CUDA 13.0
 - **Key Libraries:**
 - ultralytics (v8.4.30): For YOLOv8 model loading, training, and inference.
 - opencv-python (cv2): For image and video processing.
 - numpy and pandas: For numerical operations and data manipulation.
 - scikit-learn: For performing the stratified train/validation/test split.
 - tqdm: For displaying progress bars during data processing.
 - onnxslim and onnxruntime-gpu: For exporting the trained model to ONNX format.

4.2 Production Inference and Web Application Environment

This environment is used to run the complete PPE monitoring system after the model is trained. The specifications, as detailed in the README.md and api.py files, are as follows.

- **Hardware (Minimum Recommended):**
 - **CPU:** Modern multi-core processor (e.g., Intel Core i5-10th gen or AMD Ryzen 5 4000 series).
 - **RAM:** 8 GB (16 GB strongly recommended for smoother video processing).
 - **GPU:** An NVIDIA GPU with at least 4 GB of VRAM (e.g., GTX 1060, RTX 3050) is highly recommended for accelerated inference. The system can run on CPU only, but real-time performance will be significantly impacted.
 - **Storage:** 2 GB of free disk space for the application, the model weights (~50 MB), and stored data (incident images, videos).
- **Software Stack:**
 - **Operating System:** Windows 10/11, macOS (Big Sur or later), or Linux (Ubuntu 20.04+).
 - **Backend (FastAPI):**
 - **Language:** Python 3.10 or higher.
 - **Framework:** FastAPI (ASGI web framework for high performance).
 - **Server:** Uvicorn (ASGI server).
 - **Key Libraries:** ultralytics, opencv-python-headless (to avoid GUI dependencies), numpy, python-multipart (for file uploads), pydantic (for data validation), python-dotenv (for configuration management).
 - **Frontend (React + TypeScript):**
 - **Language:** TypeScript 5.x, HTML5, CSS3.
 - **Framework:** React 18.x (with functional components and hooks).
 - **Build Tool:** Vite 5.x (for fast development and optimized production builds).
 - **Styling:** Tailwind CSS 3.4 (utility-first CSS framework), shadcn/ui (a collection of reusable, accessible UI components built on Radix UI primitives).
 - **Key Libraries:** framer-motion (for smooth animations), lucide-react (for vector icons).
 - **Development & Deployment Tools:**
 - **Version Control:** Git (for source code management).
 - **Package Management:** pip (Python) and npm (Node.js).
 - **Web Browser:** Latest versions of Google Chrome, Mozilla Firefox, Microsoft Edge, or Apple Safari.

5. Experimental Analysis

This section provides a detailed account of the experimental setup, including dataset preparation, evaluation metrics, training results, and a comprehensive analysis of the model's performance.

5.1 Dataset Construction and Preparation

A high-quality, diverse dataset is the cornerstone of any successful deep learning project. Since a single, public dataset containing all ten required classes with consistent annotations was not readily

available, a custom dataset was constructed by aggregating and annotating images from multiple sources.

Data Sources:

- Video frames extracted from surveillance footage of actual construction sites.
- Images sourced from public internet repositories (with appropriate usage rights).
- Public datasets available on platforms like Roboflow, specifically the "Construction Site Safety Image Dataset."

Data Preprocessing and Annotation:

1. **Screening:** Collected images were manually screened to remove irrelevant background images (e.g., empty scenes with no construction personnel).
2. **Annotation:** All qualifying images were manually annotated using the labelling tool. Bounding boxes were drawn around objects belonging to the ten defined classes. The annotations were saved in the YOLO format (one .txt file per image, containing class_id x_center y_center width height for each bounding box, all normalized to image dimensions).
3. **Stratified Splitting:** To ensure a robust and unbiased evaluation, the dataset was split into training (80%), validation (10%), and test (10%) sets using a **stratified splitting strategy** based on the dominant class in each image. This is a critical step, as it ensures that the proportion of images containing rare classes (e.g., NO-Hardhat, Mask) is maintained across all three splits, preventing the model from being evaluated on an unrepresentative test set.

The final dataset statistics are presented in Table 2.

Table 2: Dataset Split and Annotation Statistics

Split	Number of Images	Number of Annotations
Train	2240	30,684
Validation	280	3,554
Test	281	4,114
Total	2801	38,352

Class Distribution (Total Annotations across all splits): The dataset exhibits a realistic, imbalanced distribution, which is typical of real-world construction site data. Person is the most frequent class, followed by machinery and NO-Safety Vest. The rarest classes are Mask and vehicle.

- Person: 9,872 (25.7%)
- machinery: 5,346 (13.9%)
- NO-Safety Vest: 4,158 (10.8%)
- Safety Cone: 3,502 (9.1%)
- Hardhat: 3,334 (8.7%)
- NO-Mask: 3,250 (8.5%)
- Safety Vest: 3,135 (8.2%)
- NO-Hardhat: 2,427 (6.3%)
- Mask: 1,700 (4.4%)

- vehicle: 1,628 (4.2%)

5.2 Evaluation Metrics

The performance of the trained YOLOv8m model was quantified using the following standard object detection metrics:

1. **Precision (P):** The ratio of correctly predicted positive instances (True Positives, TP) to all instances that were predicted as positive (TP + False Positives, FP). It answers the question: *Of all the detections the model made, how many were correct?*

$$P = TP / (TP + FP)$$

2. **Recall (R):** The ratio of correctly predicted positive instances (TP) to all actual positive instances in the ground truth (TP + False Negatives, FN). It answers the question: *Of all the objects present, how many did the model correctly find?*

$$R = TP / (TP + FN)$$

3. **F1 Score:** The harmonic mean of Precision and Recall, providing a single metric that balances both concerns. It is particularly useful when dealing with imbalanced class distributions.

$$F1 = 2 * (P * R) / (P + R)$$

4. **Intersection over Union (IoU):** A metric used to evaluate the accuracy of a predicted bounding box. It is the ratio of the area of overlap between the predicted box and the ground truth box to the area of their union. A detection is typically considered a True Positive (TP) if the IoU exceeds a defined threshold (usually 0.5).

$$IoU = \text{Area_of_Overlap} / \text{Area_of_Union}$$

5. **Mean Average Precision (mAP):** The most important metric for evaluating object detection models.

- **Average Precision (AP):** The area under the Precision-Recall curve for a single class. It summarises the precision-recall trade-off into a single number.
- **mAP@0.5:** The mean of the AP values across all classes, calculated using an IoU threshold of 0.5. This is the standard metric for assessing basic detection capability.
- **mAP@0.5:0.95:** The mean of the AP values across all classes, averaged over multiple IoU thresholds ranging from 0.5 to 0.95 (in steps of 0.05). This is a stricter metric that penalises inaccurate bounding box localisation.

5.3 Model Training Results

The YOLOv8m model was trained for 100 epochs, taking approximately 72.0 minutes on the dual Tesla T4 GPU setup. The training process was stable, with the loss values (box_loss, cls_loss, dfl_loss) steadily decreasing and the validation metrics ([mAP@0.5](#)) steadily increasing. The best model was achieved at **epoch 88**, after which the validation [mAP@0.5](#) plateaued. The results at the final epoch (100) and the best epoch (88) are summarised in Table 3.

Table 3: Training and Validation Results Summary

Metric	Epoch 100 (Final)	Best Epoch (88)
Train Box Loss	0.5954	0.7025
Train Class Loss	0.3513	0.5247
Precision	0.9131	0.9232
Recall	0.8157	0.8135

Metric	Epoch 100 (Final)	Best Epoch (88)
<u>mAP@0.5</u>	0.8739	0.8786
mAP@0.5:0.95	0.6498	0.6543

Analysis: The model shows strong performance. The final [mAP@0.5](#) of 87.4% is excellent, but the best model from epoch 88 achieves a slightly higher 87.9%. This indicates that the model began to overfit marginally after epoch 88, which is why early stopping is a standard practice. The precision (92.3%) is higher than recall (81.4%), meaning the model is more likely to miss a violation (false negative) than to raise a false alarm (false positive). This is a desirable trade-off for a safety monitoring system, as a missed violation is far more consequential than a rare false alarm.

5.4 Test Results and Per-Class Performance

The best model (from epoch 88) was evaluated on the held-out test set of 281 images (4,114 annotations), which the model had never seen during training or validation. This provides the most realistic estimate of the model's real-world performance. The final test results are presented in Table 4, and the per-class Average Precision ([AP@0.5](#)) is detailed in Table 5.

Table 4: Final Test Results

Metric	Value
<u>mAP@0.5</u>	0.8807
mAP@0.5:0.95	0.6425
Precision	0.9227
Recall	0.8092
Inference Speed	23.5 ms/image

Table 5: Per-Class Average Precision ([AP@0.5](#)) on Test Set

Class Name	<u>AP@0.5</u>	Analysis
machinery	0.9640	Excellent. Heavy machinery is large and distinct, making it easy to detect.
Mask	0.9548	Excellent. Crucial for health compliance. High performance achieved.
Person	0.9338	Excellent. Base class for most other detections. Very reliable.
vehicle	0.9057	Excellent. Vehicles are large, well-defined objects.
NO-Safety Vest	0.8917	Very Good. Violation class. Slightly lower than its compliance counterpart.
NO-Hardhat	0.8762	Very Good. Violation class. Slightly lower than its compliance counterpart.
Safety Vest	0.8505	Very Good. Can be missed if occluded or if the vest colour blends in.
NO-Mask	0.8298	Good. A difficult class as the "absence" of a mask is subtle to detect.
Hardhat	0.8276	Good. Can be confused with other headwear or missed in dense crowds.
Safety Cone	0.7725	Moderate. Most challenging class. Small size and high variability are factors.

Analysis: The model achieves excellent or very good performance ($AP > 0.85$) on 7 out of 10 classes. The performance on safety-critical classes like Mask (95.5%), NO-Safety Vest (89.2%), and NO-Hardhat (87.6%) is very high. The lower performance on Hardhat and Safety Cone can be attributed to their relatively small size in many images (hardhats are small objects) and high visual variability (safety cones come in different colours and can be partially hidden). The NO-Mask class is inherently difficult because the model must detect the absence of an object. Despite these challenges, the overall [mAP@0.5](#) of 88.07% is highly impressive and demonstrates the model's readiness for deployment.

5.5 System Performance and Functionality

The trained model was successfully integrated into the complete full-stack web application. The system demonstrates excellent functional performance.

- **Detection Speed:** The backend API, when running on a system with an NVIDIA GPU, can process a single image in approximately 23-25 milliseconds. For a video stream, this translates to a processing rate of roughly 40 frames per second, which is more than sufficient for real-time monitoring.
- **Asynchronous Video Processing:** Uploading a long video file (e.g., 10 minutes, ~60 MB) does not block the API. The endpoint returns a `job_id` immediately, and the frontend polls for status updates every 2 seconds, showing a real-time progress bar to the user. This provides a smooth and responsive user experience.
- **False Alarm Reduction (Temporal Buffer):** The 5-frame temporal buffer is highly effective. In initial tests without the buffer, transient lighting changes or motion blur would occasionally trigger false violation alerts. With the buffer enabled, the false alarm rate dropped to near zero, as a violation must be detected consistently for 5 consecutive frames (approximately 0.125 seconds of video) before an incident is logged.
- **Data Persistence:** Stopping and restarting the backend server does not result in any data loss. The `incidents.json`, `metrics.json`, and `session_state.json` files are read on startup, and all previously logged incidents, safety scores, and UI settings (like dark mode preference) are perfectly restored.
- **Frontend Responsiveness:** The React frontend is highly responsive. The live video feed (when processing a video file) updates smoothly. The safety score and metrics update dynamically. The incident log is instantly populated with new entries, and clicking on an incident displays the captured, annotated image with a clear red bounding box around the violation.

6. Discussion and Benchmarking

The results presented in the previous section confirm that the developed PPE monitoring system is both highly accurate and practical for real-world deployment. This section discusses the significance of these findings and benchmarks them against prior research.

6.1 Benchmarking Against Literature

Our work can be directly compared to several key studies in the literature.

1. **Comparison with Lightweight YOLOv5s (Li et al., 2023) [9]:** Li et al. achieved an mAP of 87.5% while reducing parameters by 54.8% and computation by 53.8% using a ShuffleNetv2 backbone. Our system achieves a slightly higher mAP of **88.07%** using the standard YOLOv8m backbone. While our model is likely larger and computationally heavier than their highly optimised version, we achieve superior accuracy. Furthermore, we have integrated our model into a complete web application, which was not a contribution of their paper. The choice of a larger model (YOLOv8m vs. a lightweight YOLOv5s) reflects a design trade-off

prioritising maximum accuracy for a server-based or workstation-based deployment, rather than edge deployment.

2. **Comparison with Comprehensive YOLO Benchmark (Elesawy et al., 2024) [11]:** This study provides the most relevant benchmark. On the CHV dataset, their best-performing model, YOLOv8m, achieved a validation [mAP@0.5](#) of **92.30%**. Our model achieves a validation [mAP@0.5](#) of **87.84%** and a test [mAP@0.5](#) of **88.07%**. There are several possible reasons for this difference:

- **Dataset Differences:** Elesawy et al. used the CHV dataset (1,330 images, 6 classes). Our dataset is larger (2,801 images) but has 10 classes, including more challenging violation classes (NO-Mask, NO-Safety Vest) that can lower the overall mAP.
- **Class Definition:** Their dataset likely had more distinct and consistently sized/appearing classes. Our dataset includes small, variable objects like Safety Cone which is our lowest-performing class.
- **Data Split:** The specific train/validation/test splits differ. A model's performance can vary by a few percentage points based on the split.

Despite these differences, our independently trained YOLOv8m model achieving an 88% mAP on a more complex, 10-class dataset is a strong result that validates the findings of Elesawy et al. and confirms YOLOv8m as a top-tier architecture for this task.

6.2 Interpretation of Key Findings

1. **YOLOv8m is Highly Effective:** The results conclusively show that YOLOv8m, with appropriate training, is capable of extremely accurate real-time PPE detection. The model's ability to achieve >95% AP on critical classes like Mask and machinery is particularly significant.
2. **Violation Detection is Feasible:** The strong performance on NO-Hardhat (87.6%) and NO-Safety Vest (89.2%) demonstrates that deep learning models can effectively detect the *absence* of required equipment. This is not a trivial task, as it requires the model to understand context.
3. **Real-World Deployment is Achievable:** The successful integration of the model into the full-stack application proves that academic research can be translated into a practical tool. The system's speed (<25ms per frame), asynchronous job processing, and state persistence are all features that are essential for real-world use but are rarely discussed in pure model comparison papers.
4. **Trade-off Between Accuracy and Lightweight Design:** The project implicitly highlights the design choices required for real systems. For a server-based system where powerful GPUs are available, a larger, more accurate model like YOLOv8m is the correct choice. For an edge deployment on a CCTV camera with an embedded GPU, a lightweight model like the one developed by Li et al. [9] would be more appropriate.

6.3 Limitations and Challenges

Despite the overall success, the system has a few limitations that should be acknowledged:

1. **Performance on Small Objects:** As evidenced by the lower AP for Safety Cone (77.2%) and Hardhat (82.8%), the model struggles more with small objects at a distance. This is a known challenge for all object detection models.
2. **Environmental Variability:** The model was not explicitly tested under extreme conditions, such as heavy rain, dense fog, or very low-light nighttime scenarios. Its performance may degrade under such conditions.

3. **Computational Requirement for Video:** While real-time processing is possible on a good GPU, processing a full day's worth of video from multiple cameras is computationally intensive and would require a scaled-up, multi-GPU setup.
4. **Dataset Bias:** While the dataset is diverse, it may still contain biases (e.g., more images of workers wearing yellow hardhats than white ones). Further expansion and diversification of the training data would improve generalisation.

7. Conclusion and Future Work

7.1 Conclusion

This research successfully developed, trained, and deployed a comprehensive, real-time Personal Protective Equipment (PPE) compliance monitoring system for the construction industry. The project addressed a critical gap in the literature by moving beyond pure model validation to deliver a complete, production-ready end-to-end solution.

A YOLOv8m object detection model was meticulously fine-tuned on a large, diverse dataset of 2,801 images with 38,352 annotations across ten classes. The model achieved a high test [mAP@0.5](#) of **88.07%**, demonstrating excellent detection capability for critical PPE items like hardhats, masks, and safety vests, as well as their corresponding violation classes. This performance is highly competitive and benchmarks strongly against recent findings in the academic literature.

This high-performance detection engine was seamlessly integrated into a robust full-stack web application. The system features an asynchronous FastAPI backend for efficient, non-blocking processing of video files, and a modern, interactive React frontend for real-time visualisation, safety scoring, incident logging, and user configuration. Key practical enhancements, including a temporal buffer for false alarm reduction, complete data persistence, and runtime model hot-swapping, were implemented to ensure the system's utility and robustness in real-world operational settings.

The system successfully balances high detection accuracy with real-time performance, processing over 40 frames per second on suitable hardware. It provides a powerful, scalable, and effective tool for construction site safety managers to proactively monitor compliance, reduce accident rates, and foster a data-driven culture of safety. The solution is particularly relevant for contractors in the Ambur –Vellore region and across India, offering a modern technological solution to a persistent and life-threatening challenge.

7.2 Future Work

While the current system is highly effective, several avenues for future enhancement and research have been identified:

1. **Edge Deployment and Optimisation:** The current system is designed for a server-based architecture. A significant next step is to optimise the model for deployment on edge devices. This would involve:
 - Converting the model to a highly optimised format like TensorRT (for NVIDIA Jetson) or OpenVINO (for Intel hardware).
 - Quantising the model to use INT8 precision, which can significantly reduce model size and inference time with a minimal accuracy trade-off.
 - Containerising the entire application (backend and frontend) using Docker and deploying it on an edge device like an NVIDIA Jetson Orin, enabling on-site, low-latency processing without relying on a central server or constant internet connectivity.
2. **Expansion of PPE Classes and Action Recognition:** The system can be extended to detect a wider range of safety equipment, including safety glasses, ear protection (earmuffs/plugs), gloves, and full-body harnesses for fall arrest. Furthermore, the system could be enhanced to

move from static object detection to action recognition. This would allow it to detect unsafe behaviours such as a worker removing their hardhat, entering a prohibited zone, or operating machinery without proper safety protocols in place.

3. **Integration with Real-Time Alerting Systems:** To maximise its impact on safety, the system can be integrated with real-time alerting mechanisms. This could involve:
 - Implementing WebSockets in the FastAPI backend to push instant notifications to the frontend when a violation occurs.
 - Integrating with a Simple Notification Service (SNS) to send SMS alerts to safety officers.
 - Connecting to a site-wide public address (PA) system or a network of smart speakers to trigger an immediate audio alert on the construction floor (e.g., "Worker near the north crane, please wear your hardhat.").
4. **Development of a Long-Term Analytics Dashboard:** The incident log and metrics provide valuable data. A future enhancement would be a dedicated analytics dashboard with features such as:
 - Historical trends of safety scores and violation rates over days, weeks, and months.
 - Identification of high-risk zones (e.g., a "heatmap" of where violations most frequently occur).
 - Automated generation of compliance reports for regulatory submissions or internal safety audits.
 - Predictive analytics to forecast potential safety incidents based on observed trends.
5. **Continuous Learning and Dataset Expansion:** To ensure the model remains robust over time, a feedback loop can be implemented. When the frontend shows a false positive or a missed detection, a user could flag it. This data could be used to create an expanded, corrected dataset for periodically retraining (fine-tuning) the model, leading to continuous improvement.

References

- [1] Construction Industry Development Council (CIDC), "Annual Report 2014," CIDC, New Delhi, India, 2014.
- [2] Korea Occupational Safety and Health Agency (KOSHA), "Occupational Safety and Health Statistics," KOSHA, Ulsan, South Korea, 2021.
- [3] H. Shi, X. Q. Chen, and Y. Yang, "An improved YOLOv3 helmet wearing detection method," *Computer Engineering and Applications*, vol. 55, no. 11, pp. 213–220, 2019.
- [4] M. J. Wei, T. Y. Zhou, Z. L. Ji, and X. N. Zhang, "A method for mask wearing detection in public places based on YOLO," *Journal of Guangxi Normal University (Natural Science)*, vol. 41, no. 01, pp. 76–86, 2023.
- [5] H. R. Cheng, X. T. Wang, J. R. Li, and W. Liu, "An improved YOLOv4-tiny pandemic collaboration mask wearing detection method," *Computer Engineering and Applications*, vol. 20, pp. 1–16, 2022.
- [6] G. B. Xie, J. J. Tang, Z. Y. Lin, X. F. Zheng, and M. Fang, "Improved YOLOv4 helmet detection algorithm in complex scenarios," *Laser Optoelectronics Progress*, vol. 20, pp. 1–13, 2022.

- [7] F. X. Song, Y. J. Wu, B. B. Liu, and Q. L. Zhang, "Safety helmet wearing detection based on improved YOLOv5s algorithm," *Computer Engineering and Applications*, vol. 59, no. 02, pp. 194–201, 2023.
- [8] Z. Wang, Y. Wu, L. Yang, A. Thirunavukarasu, C. Evison, and Y. Zhao, "Fast Personal Protective Equipment Detection for Real Construction Sites Using Deep Learning Approaches," *Sensors*, vol. 21, no. 10, p. 3478, May 2021.
- [9] S. Li, Y. Lv, X. Liu, and M. Li, "Detection of safety helmet and mask wearing using improved YOLOv5s," *Scientific Reports*, vol. 13, no. 1, p. 21417, Dec. 2023.
- [10] S. Singha, A. Jain, V. Sharma, and P. Girdhar, "PPE Detection for Construction Site Safety Leveraging YOLOv8," in *Proc. International Conference on Artificial Intelligence and Data Science (ICAIDS)*, 2024. (Note: Citation details inferred from provided file.)
- [11] A. Elesawy, E. M. Abdelkader, and H. Osman, "PPE Detection Using YOLOv5 and YOLOv8 on Construction Sites," *Buildings*, vol. 14, no. 2, p. 521, Feb. 2024.
- [12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [13] P. Mital, "PPE-Compliance-Monitor: AI-Driven PPE Compliance Monitoring System," GitHub repository, 2026. Available: PPE-Compliance-Monitor.